KVStream — A Concurrency Gateway for Microsoft Foundry Local

Technical Proposal · Revision 2

Prepared by: Shreyan Fernandes · KVStream author

Date: 25 August 2026
Status: Proposal — request for feedback

> **What changed in this revision, and why.** Revision 1 asserted a failure mode for Foundry Local — that past its optimal concurrency latency spikes and the runtime stalls. That assertion was never measured. It has now been measured on one machine, and **it was partly wrong**: the runtime serialised cleanly with no latency knee, refused nothing, and its throughput did not move between one and eight concurrent requests. A different failure did occur, and it was worse than the one predicted.
>
> This revision replaces the assumption with the measurement, narrows every claim to what the evidence supports, and leads with a reproducible defect found in Foundry Local rather than with the gateway. Section 2 is substantially rewritten. Sections 4, 5.1, 5.2, 10, 11 and 12 are revised. The architecture (6, 7) and the version-compatibility analysis (8) stand, with §8.3 now verified against a live binary.

---

1. Executive Summary

KVStream is an OpenAI-compatible gateway that sits in front of Microsoft Foundry Local and governs access to it. It resolves Foundry Local's ephemeral port, admits requests against a **measured** capacity, queues the overflow, sheds what it cannot serve, and reports when the backend has stopped working.

It runs no model, holds no KV tensors, and has no dependency on ONNX Runtime. Its value is entirely at the request layer.

This revision makes three claims, in descending order of how well they are evidenced:

1. **Foundry Local 0.8.119 can be driven into a permanent, unrecoverable stall, and its health surface reports healthy throughout.** Reproducible, measured, and described in §2.3. This is a defect report, and it is the most immediately useful thing in this document.
2. **Local inference capacity is hardware-specific and Foundry Local does not expose it.** A gateway can measure it from outside, per machine, and enforce what it measured. KVStream does this and reports what kind of runtime it found (§5.2).
3. **Admission control converts uncontrolled overload into bounded backpressure.** Measured. Whether it also raises *throughput* depends on the runtime's regime, and on the one machine measured it does not (§2.2).

What Revision 1 claimed — "users can submit far more calls without Foundry Local stalling" — is **not supported** by the evidence gathered since, and is withdrawn in that form. §2.2 explains precisely why, and §12 states what is offered instead.

---

2. The Problem, as measured

2.1 What was assumed

Revision 1 asserted: *"Past the runtime's optimal concurrency, latency spikes; beyond a threshold the runtime effectively stalls and admits no new requests."*

That is a reasonable description of many single-box runtimes. It was not measured against Foundry Local, and it should not have been stated as fact.

2.2 What was measured

`foundry 0.8.119`, model `phi-3-mini-4k` (generic-gpu variant, 2.13 GB), Windows build 19045. That build is below the 26100 Foundry Local requires to register the Windows ML execution providers, so the model ran on a CPU fallback path — stated plainly because it may be the reason for what follows.

Identical requests at rising concurrency, after a warm-up:

CONCURRENCY	WALL	MEAN LATENCY	MAX LATENCY	THROUGHPUT
1	0.38 s	0.38 s	0.38 s	2.61 req/s
2	0.77 s	0.77 s	0.77 s	2.60 req/s
4	1.55 s	1.55 s	1.55 s	2.58 req/s
8	3.12 s	3.11 s	3.12 s	2.57 req/s

Throughput flat. Latency exactly linear. Mean equal to max at every level. Zero errors. **This runtime does not batch — it serialises**, and its optimal concurrency is 1. There is no knee, nothing is refused, and there is no latency spike to prevent.

The consequence is direct and unflattering to Revision 1: **on this runtime, admission control cannot increase throughput.** 2.6 req/s is 2.6 req/s whether requests are gated or not. What a gateway can change is *what happens to the overflow* — whether it queues invisibly and unboundedly inside the runtime, or is bounded, reported and shed.

2.3 The failure that did occur

Under sustained load — 80 mixed-size requests with prompts to 1200 tokens at 16 concurrent — the runtime degraded and then **stopped serving permanently**:

- A **4-token** request did not return within 180 seconds.
- `Inference.Service.Agent` held **13.4 GB** resident for a 2.13 GB model.
- It did not recover. `foundry service restart` also hung; recovery required waiting the restart out.
- **Throughout,** `GET /v1/models` **answered** `200` **in ~4 ms.**

Two findings for the Foundry Local team, independent of whether KVStream is of any interest:

(a) The runtime can be driven into an unrecoverable state by ordinary load. This is the stall Revision 1 predicted, but the trigger is not what was predicted. It is not instantaneous concurrency — the runtime tolerated concurrency 8 indefinitely at small sizes. It is **cumulative memory growth over a session**. That distinction matters because it changes what fixes it: admission control by concurrency or by instantaneous token budget does not address it, and in one run of ours it did not prevent it.

(b) `/v1/models` is not a readiness signal, and is being used as one. A liveness endpoint that answers in 4 ms while the engine cannot produce a single token will cause every orchestrator, load balancer and container healthcheck keyed on it to route traffic into a dead backend. This is a design observation about Foundry Local's health surface, not about KVStream.

2.4 Two further observations

- **Foundry Local 0.8.119 returns no `usage` block** on chat completions. Any out-of-process component that needs token accounting must estimate, and cannot self-correct. §5.2 describes what that costs.
- **The port genuinely does move.** Across development it was observed at 64164, 51264, 51266 and 62467. This is a real and repeated operational papercut.

2.5 What this means for the premise

The honest generalisation is not "Foundry Local stalls under concurrency." It is:

> **A local runtime's capacity is a property of the model and the hardware, it is not exposed, and it cannot be assumed.**

That statement is true on every machine, and it is what the rest of this document is built on.

---

3. What KVStream Is (and Is Not)

KVSTREAM IS	KVSTREAM IS NOT
An OpenAI-compatible reverse proxy for one Foundry Local instance	An inference engine (runs no model)
An admission controller with a **measured**, not asserted, capacity	A KV-cache or tensor manager
A zero-config connector (ephemeral-port discovery)	An ONNX Runtime component or dependency
A readiness and failure detector for the backend	A multi-provider cloud gateway (LiteLLM's space)
A calibration harness that reports what kind of runtime it found	A fix for in-engine memory growth

---

4. Current State of Implementation

Revision 1's §4 is obsolete. The current state:

Implemented, tested, and exercised against a live Foundry Local:

- OpenAI-compatible proxy: `/v1/chat/completions` (streaming and non-streaming, including tool calling), `/v1/completions`, `/v1/embeddings`, `/v1/audio/transcriptions`, `/v1/models`, `/v1/models/{id}`, `/health`, `/status`, `/metrics`.
- Version-aware backend resolution: explicit URL → CLI-assisted → localhost scan → actionable failure (§8.3), verified live against the service-based CLI.
- Admission control in two modes, with strict FIFO ordering, live accounting, and predictive shedding.
- Calibration with warm-up, repeated trials, mixed request shapes, bisection refinement, and **runtime regime classification** (§5.2).

- Readiness probing, a circuit breaker, and backend drift detection.
- Response cache and request coalescing, on both streaming and non-streaming paths.
- Real Prometheus metrics; graceful drain; single-process enforcement.

364 tests, `ruff` and `mypy` clean.

Implemented but not verified on hardware:

- Token-budget admission mode. Every real-hardware run fell back to concurrency mode because the calibration record was keyed to a different device label — correct behaviour, but it means the token path has been exercised only against a modelled runtime and in unit tests.
- Foundry Local CLI 0.10.x support. No 0.10.x binary was available.
- The container image. Docker was not installed on the development machine; it has never been built or run.

Deliberately absent: the multi-machine router, semantic caching, authentication and TLS. Listed so that nothing in the tree is unwired — a static check confirms zero unreferenced definitions.

---

5. The Design

5.1 KV-Capacity Manager

Admission is by a budget the gateway holds as integer arithmetic — no KV tensors, no device memory, no ONNX Runtime. Two units share one mechanism: a request count, or a KV-token budget `B` where cost is `prompt_tokens + max_tokens`, scaled by the model's relative KV geometry.

The token budget's advantage is conditional, and Revision 1 overstated it. It exists when (a) request sizes vary, *and* (b) the runtime's capacity is actually shared across concurrent requests. On a serialising runtime neither term buys anything: the right limit is one request, and a request count expresses that perfectly well. The budget earns its keep on a runtime that batches. §5.2 is what tells an operator which they have.

Requests are admitted in strict arrival order — waking all waiters to re-check gives no ordering at all, and a large request can be starved indefinitely by a stream of small ones. Reservations are settled against the response's true length as it streams, returning headroom a short generation was never going to use. Overflow that cannot be served within its deadline is refused **on prediction rather than on elapsed time**: measured, this moved shed load from a flat 120.1 s to a median of 8.1 s.

5.2 Calibration, and what kind of runtime it found

`B` is measured, not asserted: a load sweep drives rising concurrency and watches for the knee, with warm-up, repeated trials pooled before the percentile, mixed request shapes, and bisection between the last healthy point and the first unhealthy one. The budget is stored keyed by `(model, device, CLI generation, Foundry version)`; a record from a different environment is refused rather than used, because a budget measured elsewhere is worse than none.

New in this revision: the sweep now classifies the runtime it measured.

REGIME	SIGNATURE	WHAT ADMISSION CONTROL IS WORTH
serialising	generated-token throughput flat as concurrency rises	Cannot raise throughput. Buys bounded backpressure, readiness, shedding. Cache and coalescing are the only levers that add capacity.
batching	throughput climbs to a peak, then degrades	Protects the peak; the token budget earns its keep.
capped	requests refused beyond a threshold	Converts refusals into queueing — the largest win.

Verified in both directions: a runtime modelled to batch to concurrency 4 is classified `batching, optimal 4`; the live Foundry Local is classified `serialising, optimal 1, gain 1.00x`.

This is the answer to "how does this generalise to hardware you have not tested." It does not generalise — it measures. An operator on different hardware runs the sweep and is told what their machine does, and what to expect from the gateway as a result.

> **A methodological warning, offered because it cost three attempts.** Throughput must be counted in **generated** tokens per second. Requests per second moves with the probe's request-size mix as much as with the runtime. Prompt-plus-generated tokens is worse: prefill is far cheaper per token than decode, so a mix that tilts prompt-heavy at higher concurrency manufactures a gain that is not there — it turned this project's flat runtime into a false 1.43× "batching" reading before it was caught. Anyone building similar instrumentation, inside Foundry Local or outside it, will hit this.

Token estimation. A proxy does not ship the model's tokenizer, so admission costs are estimated. Measured against a reference BPE tokenizer over 400 held-out samples: a naive `chars/4` rule under-counts 56.8% of the time; KVStream uncalibrated, 36.2%; calibrated, 16.2% with 6.8% mean absolute error. A safety factor of 1.25 eliminates under-counting at the cost of roughly 20% of admitted throughput. **Against Foundry Local 0.8.119 the calibrated row does not apply**, because that version reports no `usage` and online calibration never receives a sample.

5.3 Caching and Coalescing

Both are gated on determinism (explicit `temperature: 0`, single choice). Coalescing collapses identical concurrent requests into one upstream call on both the streaming and non-streaming paths; followers consume no admission budget.

These matter more on a serialising runtime than admission control does. Where gating cannot add capacity, removing work is the only thing that can. That inverts Revision 1's emphasis, which treated caching as a secondary optimisation.

5.4 Ephemeral-Port Discovery

Resolution is an ordered chain: explicit URL → CLI-assisted lookup → localhost scan → actionable failure. Verified live. A localhost port answering `/v1/models` is not necessarily Foundry Local, so discovery ranks candidates rather than asserting: another KVStream is excluded outright via a header it controls, a port serving the configured model wins, and an identifiable non-Foundry server is demoted.

5.5 Multi-Machine Routing (optional, not built)

Unchanged from Revision 1, and still not implemented. One machine runs one Foundry Local; a router is a fleet-only concern.

---

6. Architecture

Unchanged from Revision 1. A single co-located sidecar per Foundry Local instance is the default and, on one machine, the whole product. A thin central router is a fleet-only addition and is not built.

6.1 Request lifecycle

1. Normalise and tokenise for cost.
2. Cache lookup, if cacheable.
3. Coalesce identical in-flight calls.
4. Circuit-breaker check — fail fast if the backend is known bad.
5. Admission: reserve, or queue, or refuse on predicted wait.
6. Forward; stream back; settle the reservation live.
7. On completion: release, optionally cache, record drift and metrics.

Steps 4 and the predictive half of 5 are new since Revision 1, and both exist because of §2.3.

---

7. Topology: Sidecar vs Proxy

Unchanged. "Proxy" is a role, "sidecar" is a topology, and they compose. Co-location is not shared address space: no topology grants KVStream access to Foundry Local's KV tensors. That remains an in-process property.

---

8. Foundry Local Version Compatibility

The analysis in Revision 1 stands. Two updates:

§8.3 is verified for the service-based CLI, and required a fix. `foundry service status` reports a *status page* URL (`http://127.0.0.1:PORT/openai/status`), not an API root. Taking it literally made the CLI-assisted lookup fail every time and silently fall through to the port scan — the defensive design concealed the defect until it was run against a real binary. A reported endpoint is now tried both as given and as its bare origin. Resolution now reports `source: foundry-cli` with no scan attempted.

The 0.10.x path remains unverified. No 0.10.x binary was available. The implementation is written defensively against an unverified JSON schema and is unit-tested against synthetic output, which is not the same as working.

§8.5 stands unchanged: in-process SDK use with no web service has no insertion point for a gateway, and this is a boundary rather than a gap.

---

9. Positioning vs LiteLLM and LMCache

Unchanged. LiteLLM is a broader control plane and does not model a single device's capacity or solve Foundry's ephemeral port. LMCache is a data-plane, in-engine KV layer and is the honest realisation of KV reuse — which is direct evidence that real KV reuse is an in-engine capability, not a proxy one.

---

10. Truthfulness and Boundaries

Restated and expanded, since this is the section Revision 1's premise failed.

- No access to Foundry Local's KV tensors; no ONNX Runtime dependency.
- The budget is a **calibrated estimate**, not a memory reading.
- **KVStream does not raise a device's capacity.** On a serialising runtime it does not raise throughput at all.
- **KVStream does not prevent a runtime from exhausting itself over a session.** Measured: in one run it did not. It detects and reports the condition; it cannot stop it. Preventing it is an in-engine problem.
- **The gateway costs throughput when the backend is not overloaded** — measured at ~8% of goodput for ~12% off the tail latency.
- Caching changes response semantics and is opt-in.
- **Every quantitative claim in this document comes from one machine**, one model, one Foundry Local version, on a CPU fallback path. They are evidence, not a characterisation of Foundry Local.

---

11. Roadmap

PHASE	SCOPE	STATUS
P0	Version-aware resolution; `/v1/embeddings`, `/v1/audio/transcriptions`	**Done.** Service-based CLI verified live; 0.10.x unverified.
P1	KV-capacity manager, calibration, real metrics	**Done.** Token mode not verified on hardware.
P2	Response cache and coalescing	**Done**, exact-match only. Semantic caching not built.
P3	Fleet router, tenancy, fair queuing	Not built. Fleet-only.
P4	Upstream paged-KV / KV-offload in `onnxruntime-genai`	Not started. Separate track.

P4 is unchanged in intent and, in light of §2.3, more relevant than it was: the failure actually observed was memory growth inside the engine, which is exactly where an in-engine KV layer would act and where a gateway cannot.

---

12. What is being asked, and what is offered

Offered

1. **A reproducible defect report** (§2.3): Foundry Local 0.8.119 driven into an unrecoverable stall by ordinary sustained load, with `/v1/models` reporting healthy throughout. This is useful whether or not KVStream is.
2. **A design observation:** `/v1/models` is being used as a readiness signal and is not one.
3. **A calibration methodology** (Appendix B) that measures a runtime's capacity *and classifies its regime*, from outside the process. Reusable whether admission control ends up in a sidecar or inside `foundry server`.
4. **A working gateway** that solves the ephemeral-port papercut, admits against what it measured, and fails visibly rather than silently.

Asked

1. **A documented, stable endpoint field** in `foundry status --output json`. `FoundryLocalManager.urls` already has the value in-process; surfacing it would let every out-of-process tool retire port scanning.
2. **A `usage` block on chat completions.** Without it no out-of-process component can do honest token accounting.
3. **A readiness signal distinct from `/v1/models`** — even a bounded trial generation, or a cache-utilisation figure.
4. **Your concurrency and memory characteristics**, or a pointer to where the server enforces them. This document's numbers come from one CPU-fallback box. If Foundry Local batches on hardware with working execution providers, the regime classifier will say so and the case for admission control strengthens. **We would rather be corrected by your data than be right by assumption.**
5. **Whether this belongs in a sidecar at all.** If the team considers admission control better solved inside `foundry server`, that is a legitimate answer and we would rather hear it early. The methodology transfers either way.

Every measurement in this document is reproducible from the repository: `python benchmarks/admission_benchmark.py` for the admission comparison, `python benchmarks/estimator_benchmark.py` for the token-estimation figures, and `kvstream calibrate` for the regime classification. The gap analysis in `docs/GAP-ANALYSIS.md` records what was found, in the order it was found, including the errors.